Double-Pipe Heat Exchanger: Conjugate Heat Transfer by the Finite Difference Method

Experiment 2 | 20ME406P Computational Fluid Dynamics and Heat Transfer Lab
Name: Dobariya Darsh
Roll no.:23BME124

Aim: Predict how heat moves from a hot stream to a cold stream across a solid wall, by solving one finite-difference field for the whole cross-section — both fluids and the wall — at the same time.

Method: Pure finite differences: Taylor-series stencils on a structured grid, marched to steady state, all in this notebook. No finite-volume machinery, no external solver scripts, no VTK round-trip just to make a plot — every array below is the field actually being solved.

A real double-pipe heat exchanger is two concentric tubes: one fluid in the inner tube, the other in the annular gap, separated by the tube wall. To keep the grid a simple Cartesian one — and to keep the focus on conjugate heat transfer rather than on cylindrical geometry — we unroll that arrangement into three flat layers stacked in y:

Both channels carry a prescribed, fully-developed laminar (parabolic) velocity profile — we are solving only the energy equation here, not the momentum equation, so the flow field is an input, not an output. Everything below is where the finite-difference method does its work.

Governing equation

Assume steady, incompressible flow, constant properties within each zone, no viscous dissipation and no internal heat generation. The non-dimensional energy equation, written as one equation over the whole rectangle, is

$$u(x,y)\frac{\partial T}{\partial x} = \frac{\partial}{\partial x}\left(D(x,y)\frac{\partial T}{\partial x}\right) + \frac{\partial}{\partial y}\left(D(x,y)\frac{\partial T}{\partial y}\right)$$

With two coefficient fields that both jump at the fluid/wall interfaces:

- U(x, y) — the prescribed axial velocity: parabolic in each fluid channel, zero in the solid wall.
- D(x,y) — a diffusivity field, built from the Reynolds and Prandtl numbers and the wall/fluid conductivity ratio k_s/k_f

$D_{cold} = \frac{1}{Re_{cold}Pr}$,$D_{hot} = \frac{1}{Re_{hot}Pr}$, $D_{wall} = \frac{k_s}{k_f} . \frac{D_{hot}+D_{cold}}{2}$

Why one equation for three zones? Because u=0 turns it into pure conduction inside the wall automatically $\partial_x(D\partial_x T) + \partial_y\big(D\partial_y T\big) = 0$ with no separate equation to write and stitch on by hand. Solving one field over the whole rectangle, with U(x, y) and D(x,y) simply changing value from node to node, is what makes this a **conjugate** solution: continuity of temperature and of heat flux across the two interfaces falls out of the discretisation on its own (next section), rather than being imposed as an extra condition.

Boundary conditions

Edge	Condition	Meaning
Hot channel inlet	$T = T_{hot,in}$	Fixed inlet temperature
Hot channel inlet	$\frac{\partial T}{\partial x} = 0$	Fully developed outflow
Cold channel inlet	$T = T_{hot,in}$	Fixed inlet temperature
Cold channel inlet	$\frac{\partial T}{\partial x} = 0$	Fully developed outflow
Wall's own left & right ends	$\frac{\partial T}{\partial x} = 0$	No axial heat loss out the ends of the wall
Top & bottom of the domain	$\frac{\partial T}{\partial y} = 0$	Insulated outer surfaces

For counter-flow the cold stream enters at $x = L_x$ and exits at x=0, so its inlet/outlet columns swap sides — handled in the boundary-condition function below.

Code & Results Analysis

```
import numpy as np
import matplotlib.pyplot as plt

# Domain parameters
Lx = 30 # Length of domain
th = 0.1 # thickness of wall
Dy_hot = 5 # Diameter of each cylinder
Dy_cold = 5 # Diameter of each cylinder
Ly = Dy_hot + Dy_cold + th # Length of domain
nx = 200
ny_hot = 20
ny_wall = 3 # solid interface
ny_cold = 30
ny = ny_hot + ny_wall + ny_cold
x_array = np.linspace(0, Lx, nx)
```

```
# Each zone gets its own y-coordinates spanning ITS OWN physical
thickness
# (Dy_cold, th, Dy_hot), not the full domain height Ly. Concatenating
them
# keeps the plotted grid aligned with the true cold/wall/hot geometry.
y_cold_coords = np.linspace(0, Dy_cold, ny_cold)
y_wall_coords = np.linspace(Dy_cold, Dy_cold + th, ny_wall)
y_hot_coords  = np.linspace(Dy_cold + th, Ly, ny_hot)
y_array = np.concatenate([y_cold_coords, y_wall_coords, y_hot_coords])
X, Y = np.meshgrid(x_array, y_array)
dx = Lx/(nx-1)
# dy in each zone = that zone's own thickness / (its node count - 1),
# NOT the full domain height Ly. (Ly/(ny_wall-1) made dy_wall ~100x too
# large, which made the wall's D/dy^2 conduction term ~10,000x too weak
# and effectively decoupled the hot and cold channels.)
dy_hot = Dy_hot/(ny_hot-1)
dy_wall = th/(ny_wall-1)
dy_cold = Dy_cold/(ny_cold-1)

# Flow and Heat properties
# Water fluid and copper solid
k_f = 0.6 # thermal conductivity of water [W/mK]
k_s = 400 # thermal conductivity of copper [W/mK]
Pr = 7.0 # Approximate Prandtl number for water
Re_hot = 10 # Reynolds number of hot fluid
Re_cold = 10 # Reynolds number of cold fluid
T_hot_in = 5.0 # non-dimensional hot fluid inlet temperature
T_cold_in = -5.0 # non-dimensional cold fluid inlet temperature
dp_cold = 1.0 # pressure drop in cold channel
dp_hot = 1.0 # pressure drop in hot channel
mu_cold = 1.0 # dynamic viscosity of cold fluid
mu_hot = 1.0 # dynamic viscosity of hot fluid
counter_flow = True # counter flow if True, parallel flow if False

# Simulation parameters
ni = 2000000 # number of iterations. With this explicit (CFL-limited)
scheme,
             # dt ends up small enough that the flow needs roughly a
million+
             # steps just to advect once through the domain and let the
             # boundary layer develop - 10,000 wasn't remotely enough.
This
             # will take real wall-clock time (tens of minutes); the
tol
             # check below will still break out early if it converges
sooner.
tol = 0.5e-5 # convergence tolerance
```

```
def velocity_field(ny_cold, ny_wall, ny_hot, nx, counter_flow):
    """Parabolic (Poiseuille) profile in each channel, exactly zero in
the wall rows."""
    ny = ny_cold + ny_wall + ny_hot
    u = np.zeros((ny, nx))

    # 1. Physical coordinates measured from the centerline of each
channel (-R to +R)
    R_hot = Dy_hot / 2.0
    R_cold = Dy_cold / 2.0
    y_cold = np.linspace(-R_cold, R_cold, ny_cold)
    y_hot  = np.linspace(-R_hot, R_hot, ny_hot)

    # 2. Literal Planar Poiseuille Equation: u(y) = (ΔP / 2μL) * (R² -
y²)
    # (Denominator is 2.0 for flat planar channels, use 4.0 for
cylindrical pipes)
    profile_cold = (dp_cold / (2.0 * mu_cold * Lx)) * (R_cold**2 -
y_cold**2)
    profile_hot  = (dp_hot  / (2.0 * mu_hot * Lx))  * (R_hot**2  -
y_hot**2)

    # 3. Flow direction logic for the cold stream
    sign_cold = -1.0 if counter_flow else 1.0

    # 4. Map profiles into the global velocity array (wall rows remain
exactly zero)
    u[:ny_cold] = sign_cold * profile_cold[:, None]
    u[ny_cold + ny_wall:] = profile_hot[:, None]

    return u

u = velocity_field(ny_cold, ny_wall, ny_hot, nx, counter_flow)
v = np.zeros_like(u)  # No vertical velocity component

# # Plot the velocity field with a background color mesh and quiver
vectors
# plt.figure(figsize=(12, 5))
# # 1. Background color mesh reveals the continuous parabolic gradients
instantly
# mesh = plt.pcolormesh(X, Y, u, cmap='RdBu_r', shading='auto',
alpha=0.8)
# cbar = plt.colorbar(mesh, orientation='horizontal', pad=0.15)
# cbar.set_label('Velocity magnitude (u)')
# # 2. Sub-sample the grid (skipping points) to prevent vector
overcrowding
# # Takes every 5th vertical point and every 25th horizontal point
# skip = (slice(None, None, 5), slice(None, None, 25))
```

```
# # 3. Plot vectors using auto-scaling ('width') instead of forcing
spatial 'xy' coordinates
# plt.quiver(X[skip], Y[skip], u[skip], v[skip],
#            color='black', units='width', scale=15,
#            width=0.0015, headwidth=4)
# # 4. Draw a visual boundary for your solid copper interface wall
# plt.axhspan(Dy_cold, Dy_cold + th, color='gray', alpha=0.6,
label='Solid Interface')
# plt.title("Poiseuille Velocity Field: Parabolic Gradients & Vectors")
# plt.xlabel("Channel Length (x)")
# plt.ylabel("Channel Height (y)")
# plt.xlim(0, Lx)
# plt.ylim(0, Ly)
# plt.legend(loc='upper right')
# plt.tight_layout()
# plt.show()

# Temperature field initialization
T = np.zeros((ny, nx))
T[:ny_cold, :] = T_cold_in
T[ny_cold + ny_wall:, :] = T_hot_in

# Diffusivity field D(x,y) is piecewise constant in each region (cold,
wall, hot)
D_cold_val = 1.0 / (Re_cold * Pr)
D_hot_val = 1.0 / (Re_hot * Pr)
D_wall_val = (k_s / k_f) * ((D_hot_val + D_cold_val) / 2.0)
D = np.zeros((ny, nx))
D[:ny_cold, :] = D_cold_val
D[ny_cold:ny_cold + ny_wall, :] = D_wall_val
D[ny_cold + ny_wall:, :] = D_hot_val

def apply_bc():
    # Temperature Dirichlet boundary conditions at the inlets of the
channels
    # Temperature Neumann boundary conditions at the outlets of the
channels (zero gradient)
    # With counterflow, cold one switches direction, hot one always
goes from left to right
    if counter_flow:
        # Counter-flow: Hot enters from left (x=0), Cold enters from
right (x=Lx)
        T[:ny_cold, -1] = T_cold_in       # inlet of cold channel
        T[:ny_cold, 0] = T[:ny_cold, 1]  # outlet of cold channel
    else:
        # Co-flow (Parallel): Both fluids enter from the left edge
(x=0)
        T[:ny_cold, 0] = T_cold_in       # Left edge of cold channel
```

```
        T[:ny_cold, -1] = T[:ny_cold, -2]       # outlet of cold
channel
    T[ny_cold + ny_wall:, 0] = T_hot_in   # Left edge of hot channel
    T[ny_cold + ny_wall:, -1] = T[ny_cold + ny_wall:, -2]  # outlet of
hot channel
    # Wall Neumann boundary conditions (zero gradient) at the solid
interface
    # interface BC can be termed as continuity BC
    T[ny_cold:ny_cold + ny_wall, 0] = T[ny_cold:ny_cold + ny_wall, 1] #
Left edge of wall
    T[ny_cold:ny_cold + ny_wall, -1] = T[ny_cold:ny_cold + ny_wall, -2]
# Right edge of wall
    T[0, :] = T[1, :]  # Bottom edge of cold channel (whole domain
insulated)
    T[-1, :] = T[-2, :]  # Top edge of hot channel (whole domain
insulated)

# Create a 2D column vector of dy^2 that correctly matches the interior
grid regions (ny-2 rows)
# Because dy value is different in different regions, we need to create
a 2D array that has the correct dy values for each interior row
dy_array = np.zeros((ny - 2, 1))
dy_array[:ny_cold - 1, 0] = dy_cold
dy_array[ny_cold - 1:ny_cold + ny_wall - 1, 0] = dy_wall
dy_array[ny_cold + ny_wall - 1:, 0] = dy_hot

def fdm_step():
    Tc = T[1:-1, 1:-1]  # Interior points
    Te = T[1:-1, 2:]    # East neighbors
    Tw = T[1:-1, :-2]   # West neighbors
    Tn = T[2:, 1:-1]    # North neighbors
    Ts = T[:-2, 1:-1]   # South neighbors

    # Non-dimensionalized diffusivity D(x,y) for FDM
    # Dc = D[1:-1, 1:-1]
    # For FVM, we would have taken the harmonic mean of Dc and at half
nodes
    D_e = D[1:-1, 2:]
    D_w = D[1:-1, :-2]
    D_n = D[2:, 1:-1]
    D_s = D[:-2, 1:-1]

    diffusion_x = (D_e * (Te - Tc) - D_w * (Tc - Tw)) / dx**2
    diffusion_y = (D_n * (Tn - Tc) - D_s * (Tc - Ts)) / dy_array**2

    uc = u[1:-1, 1:-1]

    dTdx_upwind = np.where(uc >= 0, (Tc - Tw) / dx, (Te - Tc) / dx)
```

```
    convection = uc * dTdx_upwind

    # Pseudo-time stepping for the transient solution of the steady-
state problem
    # Determine maximum stable time step via Courant-Friedrichs-Lewy
(CFL) limit
    max_D = np.max(D)
    max_u = np.max(np.abs(uc))
    min_dy = min(dy_cold, dy_wall, dy_hot)
    dt = 0.4 / (max_D * (2/dx**2 + 2/min_dy**2) + max_u/dx)

    T_new = T.copy()
    T_new[1:-1, 1:-1] = Tc + dt * (diffusion_x + diffusion_y -
convection)

    return T_new

######################
# Main simulation loop
apply_bc()

for it in range(ni):
    T_old = T.copy()
    T = fdm_step()

    apply_bc()  # Re-apply boundary conditions after updating interior
points

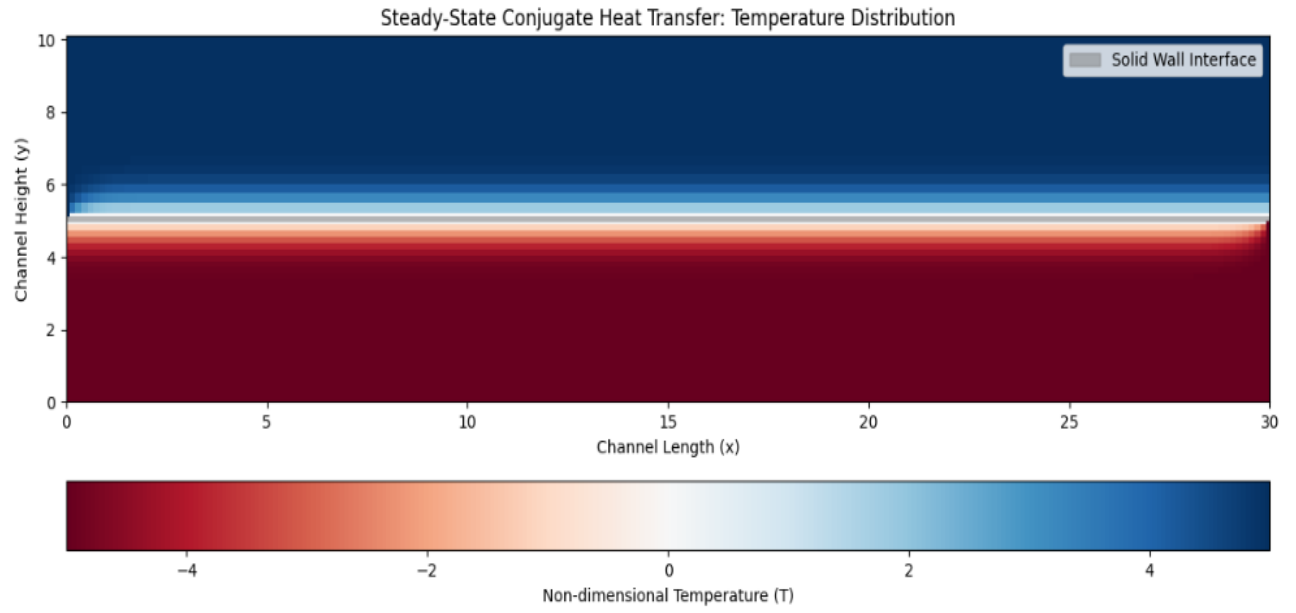
    # Compute maximum absolute change for convergence
    error = np.max(np.abs(T - T_old))

    if it % 500 == 0 or it == ni - 1:
        print(f"Iteration {it}, Error: {error:.6e}")

    if error < tol:
        print(f"Converged successfully at iteration {it} with error
{error:.6e}")
        break

# Plotting the Final Temperature Field
plt.figure(figsize=(12, 5))
mesh = plt.pcolormesh(X, Y, T, cmap='RdBu', shading='auto')
cbar = plt.colorbar(mesh, orientation='horizontal', pad=0.15)
cbar.set_label('Non-dimensional Temperature (T)')
plt.axhspan(Dy_cold, Dy_cold + th, color='gray', alpha=0.5,
label='Solid Wall Interface')
plt.title("Steady-State Conjugate Heat Transfer: Temperature
Distribution")
```

```
plt.xlabel("Channel Length (x)")
plt.ylabel("Channel Height (y)")
plt.xlim(0, Lx)
plt.ylim(0, Ly)
plt.legend(loc='upper right')
plt.tight_layout()
plt.show()
```



1. **2D Steady-State Temperature Field (Re = 10)**
   The 2D spatial contour plot displays the steady-state thermal distribution established between the cold bottom channel, central copper interface wall, and hot top channel:
   - **Thermal Boundary Layer Development**: Significant vertical temperature gradients are concentrated strictly within thin fluid boundary layers adjacent to the solid interface ($y \approx 4\ to\ 6$).
   - **Unaffected Core Channels**: Because Pr = 7.0 (water) yields moderate thermal diffusivity relative to advection, the bulk core fluids near the top ($y \approx 10$) and bottom ($y \approx 0$) boundary boundaries remain near their original inlet values ($T \approx +5\ and\ T \approx -5$)
   - **Negligible Solid Thermal Resistance:** Owing to the extremely high thermal conductivity ratio of copper to water (k_s/k_f ≈666.7), the temperature drop across the solid wall (y ∈ [5.0, 5.1]) is minimal. The solid interface acts as an almost ideal heat conductor, shifting the primary bottleneck for heat transfer entirely to convective resistance within the fluid boundary layers.

Parametric Study

```
import numpy as np
from matplotlib import pyplot as plt
import concurrent.futures
import os

#########################################################################
#
# PARAMETRIC STUDY: sweep any single scalar parameter and compare the
# ny-averaged temperature profile T_avg(x) for the cold and hot
channels.
#
# Nothing above this line is touched. Everything below is self-
contained:
# each call rebuilds its own grid, velocity field, diffusivity field
and
# runs its own FDM loop to convergence, so sweeping a parameter here
never
# mutates the baseline T, D, u, etc. computed above.
#########################################################################
#

# Domain parameters
Lx = 30 # Length of domain
th = 0.1 # thickness of wall
Dy_hot = 5 # Diameter of each cylinder
Dy_cold = 5 # Diameter of each cylinder
Ly = Dy_hot + Dy_cold + th # Length of domain
nx = 200 # reduced from 1000: dx barely affects the CFL limit here (the
wall
          # does), so a finer nx just burns runtime without buying
stability;
          # bump this back up once you've confirmed convergence
ny_hot = 100
ny_wall = 3 # solid interface. Reduced from 10: pure conduction across
a thin
            # wall is a LINEAR profile, so 3 points resolve it exactly.
This
            # was the main thing forcing a tiny, CFL-limited dt (see
fdm_step)
ny_cold = 100
ny = ny_hot + ny_wall + ny_cold
x_array = np.linspace(0, Lx, nx)
# Each zone gets its own y-coordinates spanning ITS OWN physical
thickness
# (Dy_cold, th, Dy_hot), not the full domain height Ly. Concatenating
them
# keeps the plotted grid aligned with the true cold/wall/hot geometry.
```

```
y_cold_coords = np.linspace(0, Dy_cold, ny_cold)
y_wall_coords = np.linspace(Dy_cold, Dy_cold + th, ny_wall)
y_hot_coords  = np.linspace(Dy_cold + th, Ly, ny_hot)
y_array = np.concatenate([y_cold_coords, y_wall_coords, y_hot_coords])
X, Y = np.meshgrid(x_array, y_array)
dx = Lx/(nx-1)
# dy in each zone = that zone's own thickness / (its node count - 1),
# NOT the full domain height Ly. (Ly/(ny_wall-1) made dy_wall ~100x too
# large, which made the wall's D/dy^2 conduction term ~10,000x too weak
# and effectively decoupled the hot and cold channels.)
dy_hot = Dy_hot/(ny_hot-1)
dy_wall = th/(ny_wall-1)
dy_cold = Dy_cold/(ny_cold-1)

# Flow and Heat properties
# Water fluid and copper solid
k_f = 0.6 # thermal conductivity of water [W/mK]
k_s = 400 # thermal conductivity of copper [W/mK]
Pr = 7.0 # Approximate Prandtl number for water
Re_hot = 100 # Reynolds number of hot fluid
Re_cold = 100 # Reynolds number of cold fluid
T_hot_in = 1.0 # non-dimensional hot fluid inlet temperature
T_cold_in = 0.0 # non-dimensional cold fluid inlet temperature
dp_cold = 1.0 # pressure drop in cold channel
dp_hot = 1.0 # pressure drop in hot channel
mu_cold = 1.0 # dynamic viscosity of cold fluid
mu_hot = 1.0 # dynamic viscosity of hot fluid
counter_flow = True # counter flow if True, parallel flow if False

# Simulation parameters
ni = 2000000 # number of iterations. With this explicit (CFL-limited)
scheme,
             # dt ends up small enough that the flow needs roughly a
million+
             # steps just to advect once through the domain and let the
             # boundary layer develop - 10,000 wasn't remotely enough.
This
             # will take real wall-clock time (tens of minutes); the
tol
             # check below will still break out early if it converges
sooner.
tol = 0.5e-5 # convergence tolerance

def run_case(param_name, param_value, ni_local=None, tol_local=None,
verbose=False):
    """
    Run one full conjugate-heat-transfer simulation with a single named
```

```
    parameter overridden from the baseline (module-level) values
defined
    at the top of this script; every other parameter keeps its baseline
    value.

    Parameters
    ----------
    param_name : str
        Name of the parameter to override. One of:
        'Lx', 'th', 'Dy_hot', 'Dy_cold', 'nx', 'ny_hot', 'ny_wall',
'ny_cold',
        'k_f', 'k_s', 'Pr', 'Re_hot', 'Re_cold', 'T_hot_in',
'T_cold_in',
        'dp_cold', 'dp_hot', 'mu_cold', 'mu_hot', 'counter_flow'.
    param_value : any
        Value to use for param_name in this run.
    ni_local, tol_local : optional
        Override the baseline `ni` / `tol` for this run (useful to cap
        runtime during a sweep without touching the baseline single
run).
    verbose : bool
        Print convergence progress for this run.

    Returns
    -------
    x_arr : (nx,) array
    T_avg_cold : (nx,) array, T averaged over all ny_cold rows at each
x
    T_avg_hot  : (nx,) array, T averaged over all ny_hot rows at each x
    """
    baseline = dict(
        Lx=Lx, th=th, Dy_hot=Dy_hot, Dy_cold=Dy_cold,
        nx=nx, ny_hot=ny_hot, ny_wall=ny_wall, ny_cold=ny_cold,
        k_f=k_f, k_s=k_s, Pr=Pr, Re_hot=Re_hot, Re_cold=Re_cold,
        T_hot_in=T_hot_in, T_cold_in=T_cold_in,
        dp_cold=dp_cold, dp_hot=dp_hot, mu_cold=mu_cold, mu_hot=mu_hot,
        counter_flow=counter_flow,
        ni=ni_local if ni_local is not None else ni,
        tol=tol_local if tol_local is not None else tol,
    )
    if param_name not in baseline:
        raise ValueError(f"Unknown parameter '{param_name}'. Choose
from: {sorted(baseline.keys())}")
    baseline[param_name] = param_value
    p = baseline  # shorthand

    Ly_ = p['Dy_hot'] + p['Dy_cold'] + p['th']
    ny_ = p['ny_hot'] + p['ny_wall'] + p['ny_cold']
```

```
    x_arr = np.linspace(0, p['Lx'], p['nx'])
    dx_ = p['Lx'] / (p['nx'] - 1)
    dy_hot_ = p['Dy_hot'] / (p['ny_hot'] - 1)
    dy_wall_ = p['th'] / (p['ny_wall'] - 1)
    dy_cold_ = p['Dy_cold'] / (p['ny_cold'] - 1)

    # --- Velocity field (same Poiseuille logic as velocity_field()
above) ---
    u_ = np.zeros((ny_, p['nx']))
    R_hot = p['Dy_hot'] / 2.0
    R_cold = p['Dy_cold'] / 2.0
    y_cold_ = np.linspace(-R_cold, R_cold, p['ny_cold'])
    y_hot_ = np.linspace(-R_hot, R_hot, p['ny_hot'])
    profile_cold = (p['dp_cold'] / (2.0 * p['mu_cold'] * p['Lx'])) *
(R_cold**2 - y_cold_**2)
    profile_hot = (p['dp_hot'] / (2.0 * p['mu_hot'] * p['Lx'])) *
(R_hot**2 - y_hot_**2)
    sign_cold = -1.0 if p['counter_flow'] else 1.0
    u_[:p['ny_cold']] = sign_cold * profile_cold[:, None]
    u_[p['ny_cold'] + p['ny_wall']:] = profile_hot[:, None]

    # --- Temperature field init ---
    T_ = np.zeros((ny_, p['nx']))
    T_[:p['ny_cold'], :] = p['T_cold_in']
    T_[p['ny_cold'] + p['ny_wall']:, :] = p['T_hot_in']

    # --- Diffusivity field ---
    D_cold_val_ = 1.0 / (p['Re_cold'] * p['Pr'])
    D_hot_val_ = 1.0 / (p['Re_hot'] * p['Pr'])
    D_wall_val_ = (p['k_s'] / p['k_f']) * ((D_hot_val_ + D_cold_val_) /
2.0)
    D_ = np.zeros((ny_, p['nx']))
    D_[:p['ny_cold'], :] = D_cold_val_
    D_[p['ny_cold']:p['ny_cold'] + p['ny_wall'], :] = D_wall_val_
    D_[p['ny_cold'] + p['ny_wall']:, :] = D_hot_val_

    ny_cold_, ny_wall_, ny_hot_ = p['ny_cold'], p['ny_wall'],
p['ny_hot']
    T_cold_in_, T_hot_in_, counter_flow_ = p['T_cold_in'],
p['T_hot_in'], p['counter_flow']

    def apply_bc_local(Tf):
        if counter_flow_:
            Tf[:ny_cold_, -1] = T_cold_in_
            Tf[:ny_cold_, 0] = Tf[:ny_cold_, 1]
        else:
            Tf[:ny_cold_, 0] = T_cold_in_
            Tf[:ny_cold_, -1] = Tf[:ny_cold_, -2]
```

```
        Tf[ny_cold_ + ny_wall_:, 0] = T_hot_in_
        Tf[ny_cold_ + ny_wall_:, -1] = Tf[ny_cold_ + ny_wall_:, -2]
        Tf[ny_cold_:ny_cold_ + ny_wall_, 0] = Tf[ny_cold_:ny_cold_ +
ny_wall_, 1]
        Tf[ny_cold_:ny_cold_ + ny_wall_, -1] = Tf[ny_cold_:ny_cold_ +
ny_wall_, -2]
        Tf[0, :] = Tf[1, :]
        Tf[-1, :] = Tf[-2, :]
        return Tf

    dy_array_ = np.zeros((ny_ - 2, 1))
    dy_array_[:ny_cold_ - 1, 0] = dy_cold_
    dy_array_[ny_cold_ - 1:ny_cold_ + ny_wall_ - 1, 0] = dy_wall_
    dy_array_[ny_cold_ + ny_wall_ - 1:, 0] = dy_hot_

    def fdm_step_local(Tf):
        Tc = Tf[1:-1, 1:-1]
        Te = Tf[1:-1, 2:]
        Tw = Tf[1:-1, :-2]
        Tn = Tf[2:, 1:-1]
        Ts = Tf[:-2, 1:-1]
        D_e = D_[1:-1, 2:]
        D_w = D_[1:-1, :-2]
        D_n = D_[2:, 1:-1]
        D_s = D_[:-2, 1:-1]
        diffusion_x = (D_e * (Te - Tc) - D_w * (Tc - Tw)) / dx_**2
        diffusion_y = (D_n * (Tn - Tc) - D_s * (Tc - Ts)) /
dy_array_**2
        uc = u_[1:-1, 1:-1]
        dTdx_upwind = np.where(uc >= 0, (Tc - Tw) / dx_, (Te - Tc) /
dx_)
        convection = uc * dTdx_upwind
        max_D = np.max(D_)
        max_u = np.max(np.abs(uc))
        min_dy = min(dy_cold_, dy_wall_, dy_hot_)
        dt = 0.4 / (max_D * (2 / dx_**2 + 2 / min_dy**2) + max_u / dx_)
        Tf_new = Tf.copy()
        Tf_new[1:-1, 1:-1] = Tc + dt * (diffusion_x + diffusion_y -
convection)
        return Tf_new

    T_ = apply_bc_local(T_)
    for it in range(p['ni']):
        T_old_ = T_.copy()
        T_ = fdm_step_local(T_)
        T_ = apply_bc_local(T_)
        error_ = np.max(np.abs(T_ - T_old_))
        if verbose and (it % 2000 == 0):
```

```
            print(f"[{param_name}={param_value}] iter {it}, error
{error_:.3e}")
        if error_ < p['tol']:
            if verbose:
                print(f"[{param_name}={param_value}] converged at iter
{it}, error {error_:.3e}")
            break

    T_avg_cold = T_[:ny_cold_, :].mean(axis=0)
    T_avg_hot = T_[ny_cold_ + ny_wall_:, :].mean(axis=0)

    return x_arr, T_avg_cold, T_avg_hot


def parametric_study(param_name, param_values, ni_local=None,
tol_local=None, verbose=True):
    """
    Run run_case() once per value in param_values (sweeping param_name)
and
    plot the ny-averaged temperature T_avg(x) for the cold and hot
channels
    side by side, one curve per parameter value.

    SEQUENTIAL version - runs values one after another in this process.
    See parametric_study_parallel() below for the multi-process
version.
    """
    results = {}
    for val in param_values:
        x_arr, T_avg_cold, T_avg_hot = run_case(
            param_name, val, ni_local=ni_local, tol_local=tol_local,
verbose=verbose
        )
        results[val] = (x_arr, T_avg_cold, T_avg_hot)

    _plot_parametric_results(param_name, param_values, results,
suffix="")
    return results


#########################################################################
#
# PARALLEL VERSION
#
# Each run_case() call is fully independent - it builds its own u, D, T
# arrays and runs its own loop - so the sweep is embarrassingly
parallel
# across the different parameter values. There is no shared state
between
```

```
# runs, so no locks/synchronization are needed.
#
# This uses ProcessPoolExecutor (separate OS processes), not threads.
# NumPy releases the GIL for individual large array ops, but
fdm_step_local
# is a tight Python for-loop doing many small-to-medium array ops per
# iteration (this domain is only 206x203), so under plain threading the
# GIL would still serialize most of the work and you'd see little to no
# speedup. Separate processes give real parallelism across CPU cores at
# the cost of a bit of IPC overhead (pickling the small return arrays).
#############################################################################
#

def _run_case_worker(args):
    """Picklable top-level wrapper so run_case can be sent to a worker
process."""
    param_name, param_value, ni_local, tol_local, verbose = args
    return param_value, run_case(param_name, param_value,
                                 ni_local=ni_local,
tol_local=tol_local, verbose=verbose)


def parametric_study_parallel(param_name, param_values, ni_local=None,
tol_local=None,
                              verbose=False, max_workers=None):
    """
    Same as parametric_study(), but dispatches each parameter value's
    simulation to its own process via
concurrent.futures.ProcessPoolExecutor,
    so independent Re (or any other parameter) cases solve in parallel
    across CPU cores instead of one after another.

    max_workers defaults to os.cpu_count(). Must be called from inside
    `if __name__ == "__main__":` (see bottom of this file) - this is a
    hard requirement of the multiprocessing module, not optional style.
    """
    max_workers = max_workers or os.cpu_count()
    tasks = [(param_name, val, ni_local, tol_local, verbose) for val in
param_values]

    results = {}
    with
concurrent.futures.ProcessPoolExecutor(max_workers=max_workers) as
executor:
        for val, (x_arr, T_avg_cold, T_avg_hot) in
executor.map(_run_case_worker, tasks):
            results[val] = (x_arr, T_avg_cold, T_avg_hot)

    _plot_parametric_results(param_name, param_values, results,
```

```
                                suffix=f" (parallel, {max_workers}
workers)")
    return results


def _plot_parametric_results(param_name, param_values, results,
suffix=""):
    """Shared plotting code for both the sequential and parallel sweep
functions."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)
    colors = plt.cm.viridis(np.linspace(0, 1, len(param_values)))

    for color, val in zip(colors, param_values):
        x_arr, T_avg_cold, T_avg_hot = results[val]
        axes[0].plot(x_arr, T_avg_cold, color=color,
label=f"{param_name} = {val}")
        axes[1].plot(x_arr, T_avg_hot, color=color,
label=f"{param_name} = {val}")

    axes[0].set_title("Cold channel: y-averaged temperature")
    axes[0].set_xlabel("x")
    axes[0].set_ylabel("Average T")
    axes[0].legend()
    axes[0].grid(alpha=0.3)

    axes[1].set_title("Hot channel: y-averaged temperature")
    axes[1].set_xlabel("x")
    axes[1].legend()
    axes[1].grid(alpha=0.3)

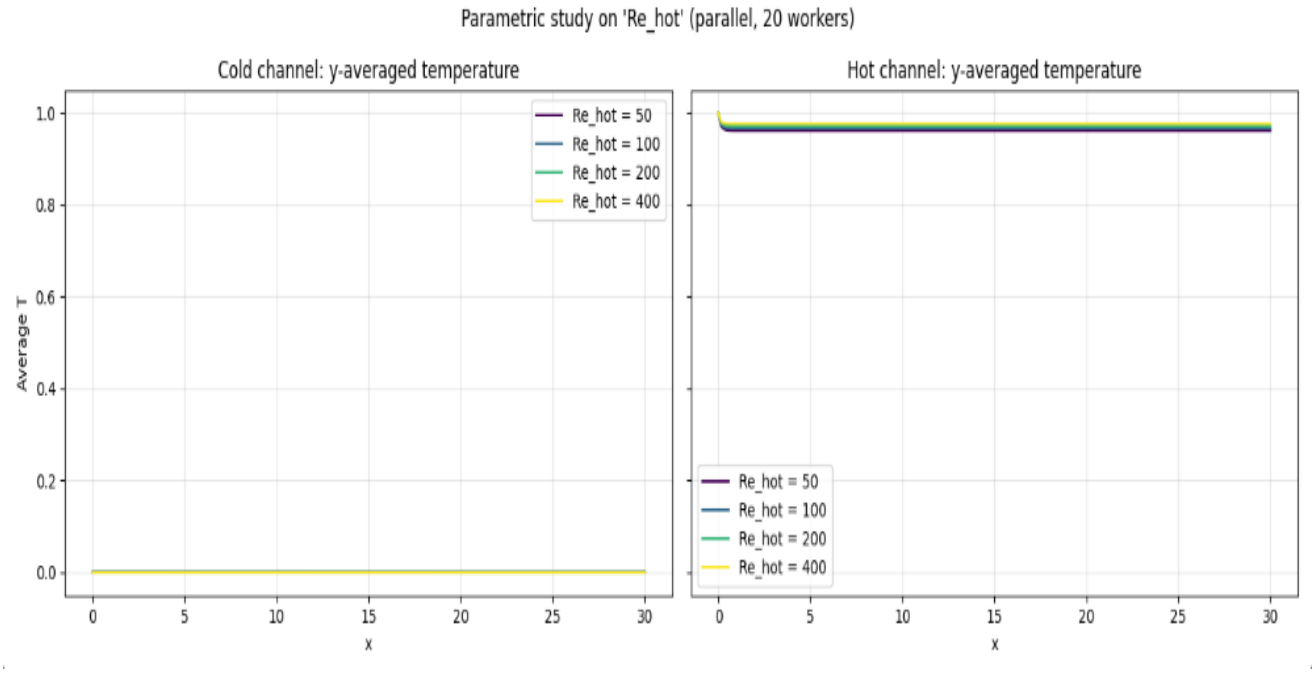
    fig.suptitle(f"Parametric study on '{param_name}'{suffix}")
    plt.tight_layout()
    plt.show()


# --- Example usage ----------------------------------------------------
-----
# Wrapped in `if __name__ == "__main__":` because
parametric_study_parallel()
# uses ProcessPoolExecutor: worker processes re-import this module, and
# without the guard they would re-execute every top-level statement
# (including spawning their own pools) instead of just defining
functions.
# This guard is required for correctness, not just style, on any
platform
# using the 'spawn' start method (default on Windows/macOS), and is
good
# practice even under Linux's default 'fork'.
if __name__ == "__main__":
```

```
    # Sequential sweep (same as before)
    # results = parametric_study('Re_hot', [50, 100, 200, 400],
    #                            ni_local=300000, tol_local=1e-5)

    # Parallel sweep - identical results, cases solved concurrently
across
    # CPU cores instead of one after another
    results_parallel = parametric_study_parallel('Re_hot', [50, 100,
200, 400],
                                                 ni_local=300000,
tol_local=1e-5)

    # To sweep a different variable, just change the name and values,
e.g.:
    # results = parametric_study_parallel('dp_hot', [0.5, 1.0, 2.0,
4.0],
    #                                     ni_local=300000,
tol_local=1e-5)
```



2. Parametric Study: Effect of Reynolds Number ($Re_{hot} \in [50, 400]$)

The y-averaged channel temperature profiles $T_{avg}(x)$ across Re_{hot} = 50, 100, 200, and 400 demonstrate the strong role of advective transport:

- **Advective Dominance:** As Re increases, the Peclet number ($Pe = Re.Pr$) scales upward (Pe≥ 350). Stronger axial advection reduces the residence time of the fluid inside the channel.
- **Suppression of Transverse Penetration:** Due to the reduced contact/residence time at higher Re, heat from the hot channel penetrates less deeply into the bulk fluid cross-section per unit length:

    - The **Cold Channel** retains a near-constant mean temperature $T_{avg,cold}(x) = 0$
    - The **Hot Channel** experiences a slight initial temperature drop near the entrance (x= 0) before leveling off at $T_{avg,hot}(x) = 0.96\text{-}0.98$
- **Effectiveness vs. Re Trade-off:** Increasing Re reduces the individual stream temperature change (ΔT), leading to lower heat exchanger effectiveness. However, the higher mass flow rate increases the overall integrated thermal energy transferred per unit time.

Conclusion

1. **Dominant Resistance:** The highly conductive solid interface (k_s/k_f ≈666.7) contributes virtually no conduction resistance. Consequently, total heat exchanger performance is governed almost exclusively by fluid-side convective boundary layers.
2. **Impact of Flow Rate (Re):** Operating at higher Reynolds numbers (Re $\geq$ 50) shifts the thermal transport regime heavily toward axial advection. This decreases fluid residence time, resulting in small bulk temperature changes (ΔT) along the domain length L_x = 30.
3. **Design Implication:** To achieve significant bulk fluid temperature cross-over or higher effectiveness, heat exchangers operating at high Re require either significantly longer channel lengths ($L_x \gg 30$), narrower channel heights (D_y < 5), or boundary-layer mixing enhancements (e.g., internal ribs or turbulators).